



Politecnico
di Torino

Rakuten
Mobile

eBPF-Based Runtime Security Monitoring System

End-to-End Pipeline and Hook Implementation

Simone Romano · Giuseppe Insalaco

Project Reference and Architectural Constraints



Tracee - Reference

- General-purpose, multi-kernel runtime security.
- Ring buffer as the primary transport channel.
- Reference for the probe registry, pipeline, and event model.

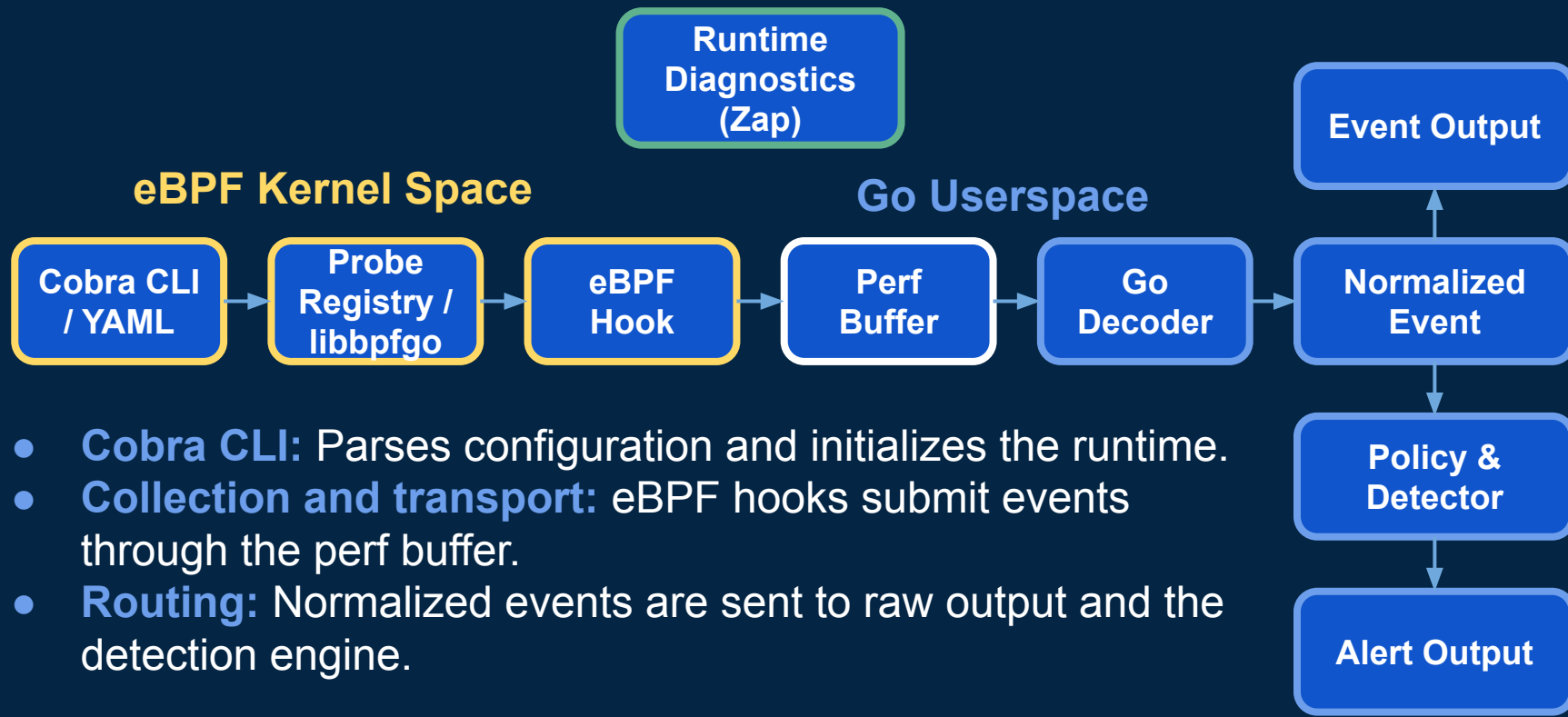


Our Approach

- Targeted at Rocky Linux 8, kernel 4.18.
- Adapted to verifier constraints and provable memory offsets.
- Perf buffer as the main operational transport.
- Lightweight policy layer.

Not a Tracee replica, but a more focused and controllable runtime.

End-to-End Architecture and Pipeline



Binary Contract and Event Normalization

Kernel
[event_context_t + argnum + args buffer]

From binary records

Shared C/Go Registry

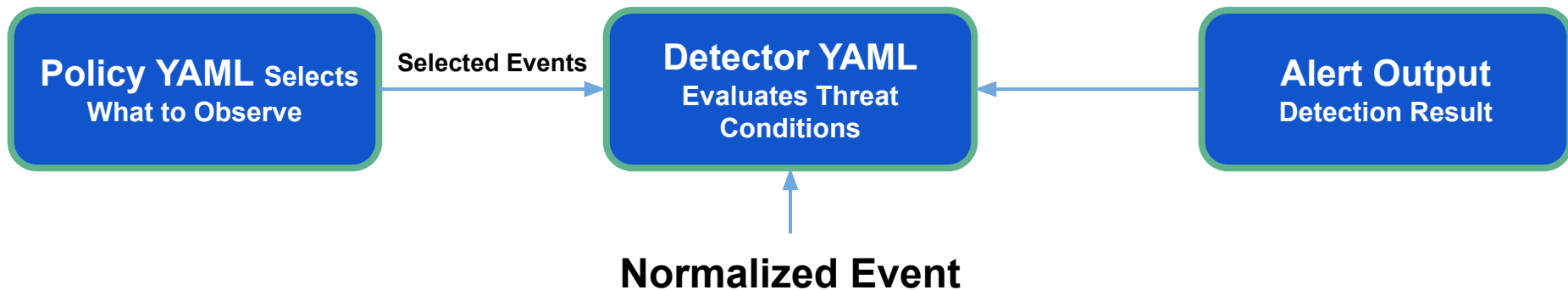
to typed, indexed arguments

Userspace

Event:	security_file_open
Path:	/etc/shadow
Flag:	O_RDONLY

- **Binary contract:** Shared context, indexed arguments, and synchronized C/Go registries.
- **Unique mapping:** eBPF event ID, event name, and decoding schema.
- **Enrichment:** Userspace normalization of capabilities, flags, bitmasks, and errno values.

Policy and Detection Layer (YAML)

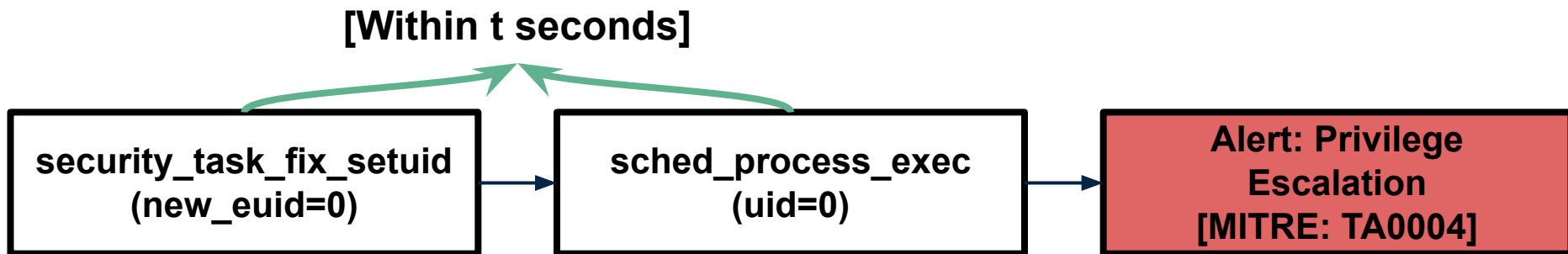


- **Policy:** Defines the monitoring scenario and selects the required events.
- **Detector:** Evaluates declarative conditions over normalized events.
- **Alert:** Represents the detection outcome delivered to the analyst.

Policies and detectors are validated before loading and attaching eBPF programs.

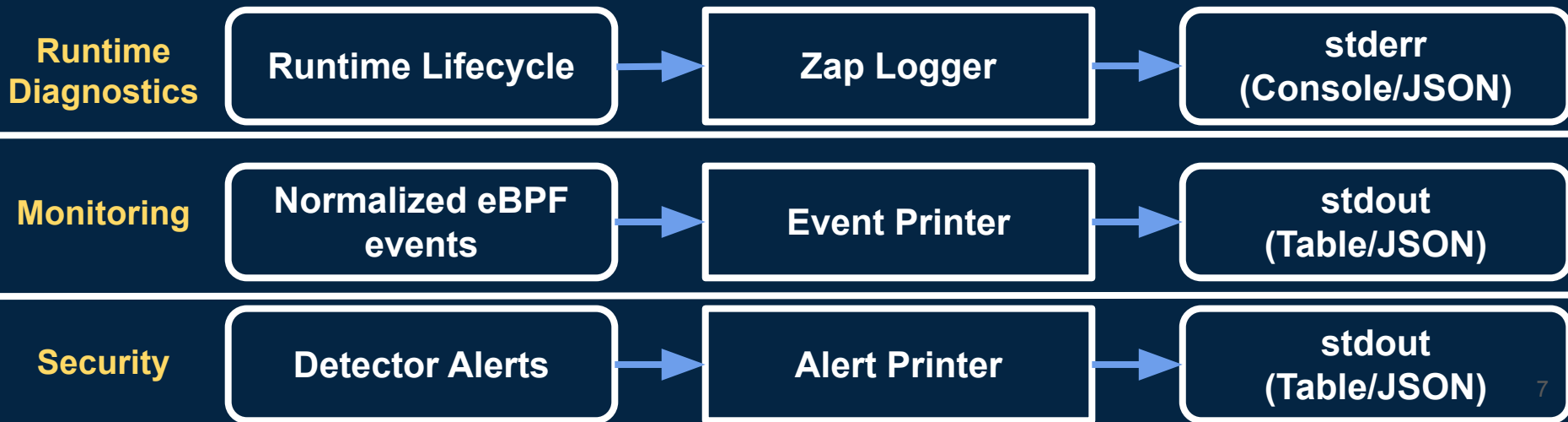
Point and Collective Detection (MITRE ATT&CK)

- **Point Anomaly:** A condition evaluated on a single isolated event.
- **Collective Anomaly:** Temporal correlation of a short event sequence.
- **Engine & Dedup:** A five-second window reduces repeated alerts.
- **MITRE Metadata:** Tactics and techniques enrich alerts with security context.



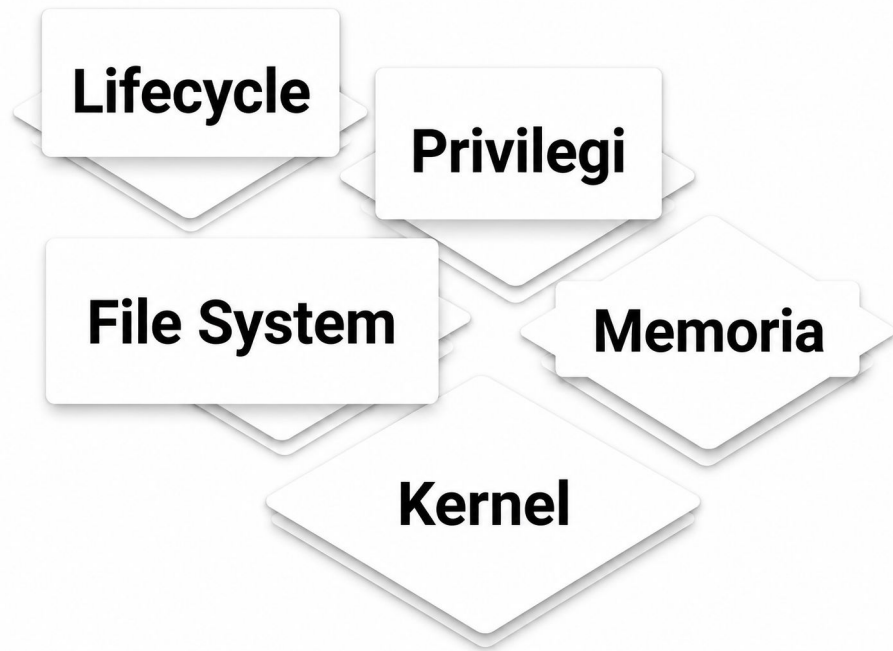
Configuration, Logging, and Output Routing

- **Clear separation:** Zap handles runtime lifecycle logs, while dedicated printers format events and alerts.
- **--alerts-only mode:** Hides raw terminal output while keeping events available to detectors.
- **Kubernetes readiness:** Independent streams support containerized and centralized deployments.



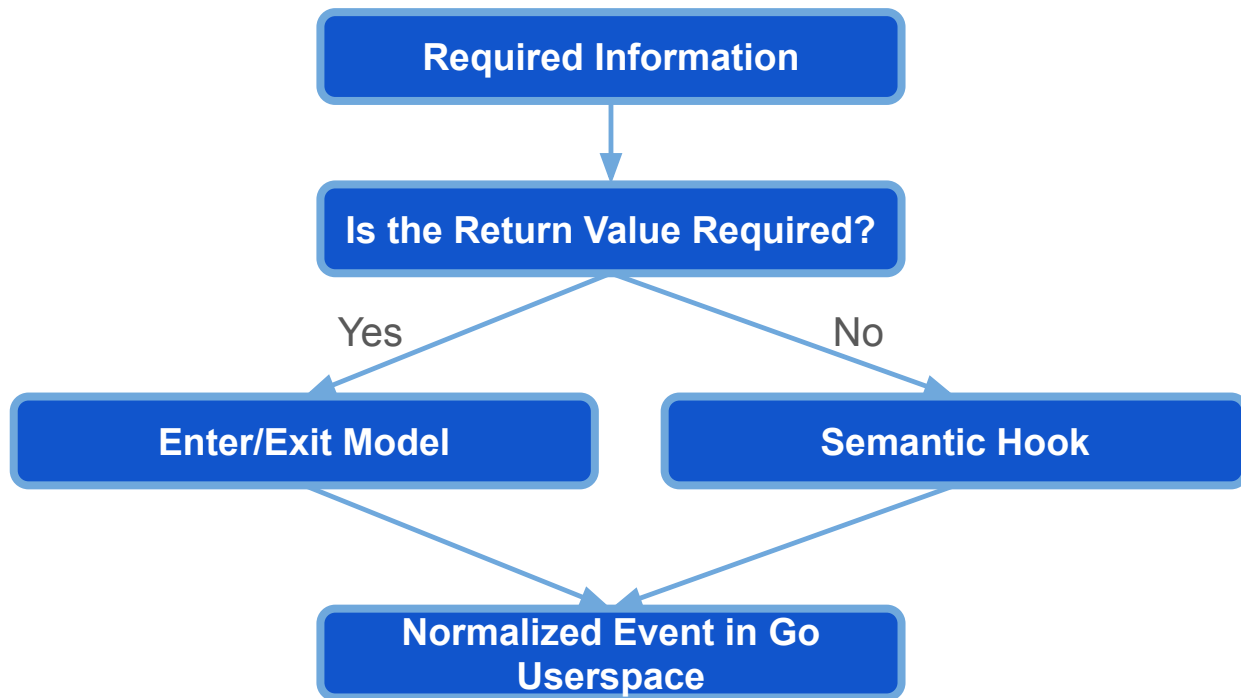
eBPF Process Monitoring

- **Coverage strategy:** Prefer semantic kprobes and security hooks over noisy generic tracepoints.
- **Return-value tracking:** Apply the enter/exit model only when the syscall outcome is required.
- **Objective:** Targeted host-level monitoring, excluding the network layer.

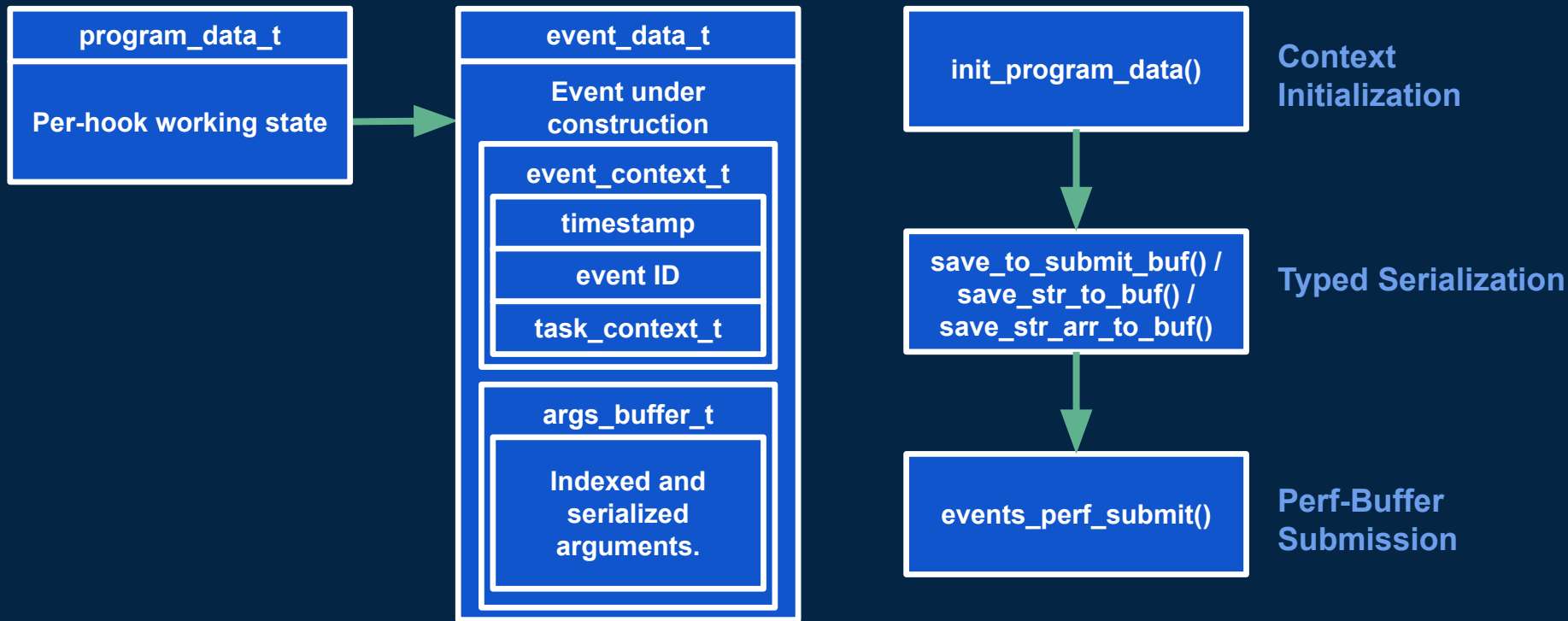


Hook Implementation Strategy

- **Semantic hooks:**
Primarily based on kprobes and security hooks exposing resolved kernel objects.
- **Enter/exit model:** Entry stores arguments; exit adds the final return value.
- **Kernel adaptation:**
Designed around Rocky Linux 4.18 verifier constraints.



Common eBPF Implementation Pattern



When the return value is required, entry stores the arguments and exit completes the event with the syscall outcome.

Process Lifecycle

fork / vfork / clone
Syscall Request:
Capture the initial
process-creation request.

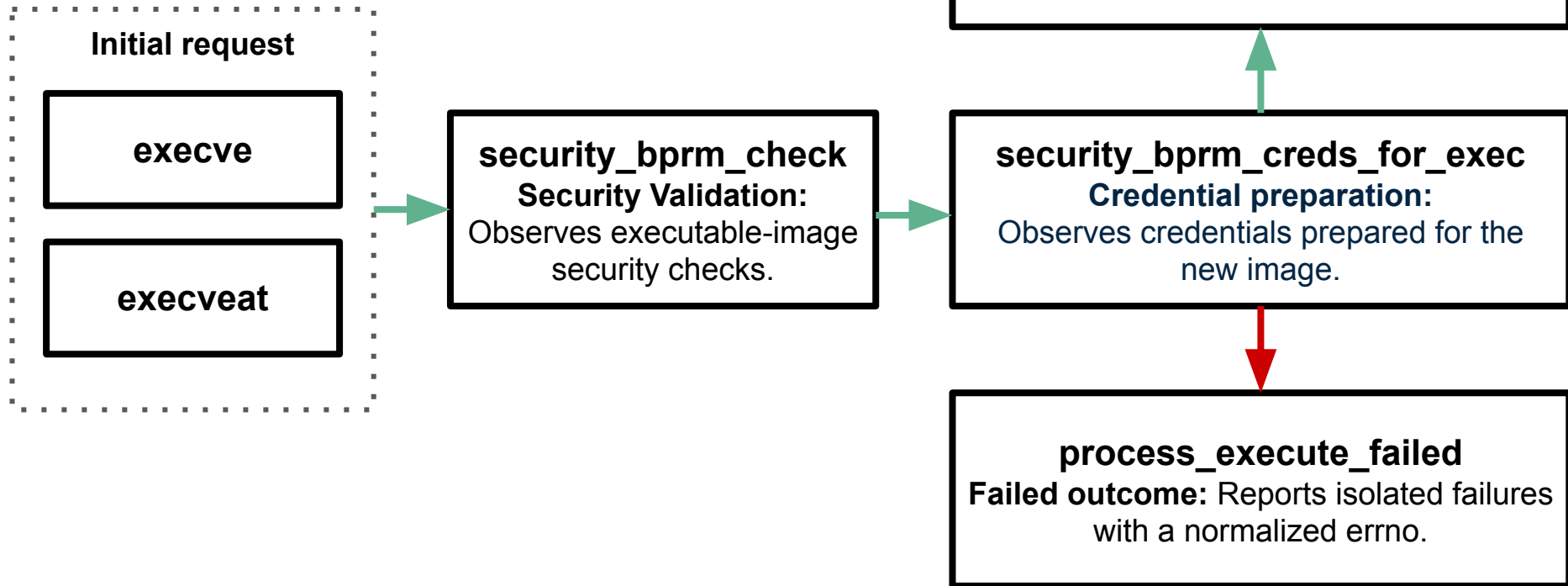
task_rename
Task transition:
Track changes to the
process comm.

sched_process_exit
Termination:
Record process exit and
exit code.

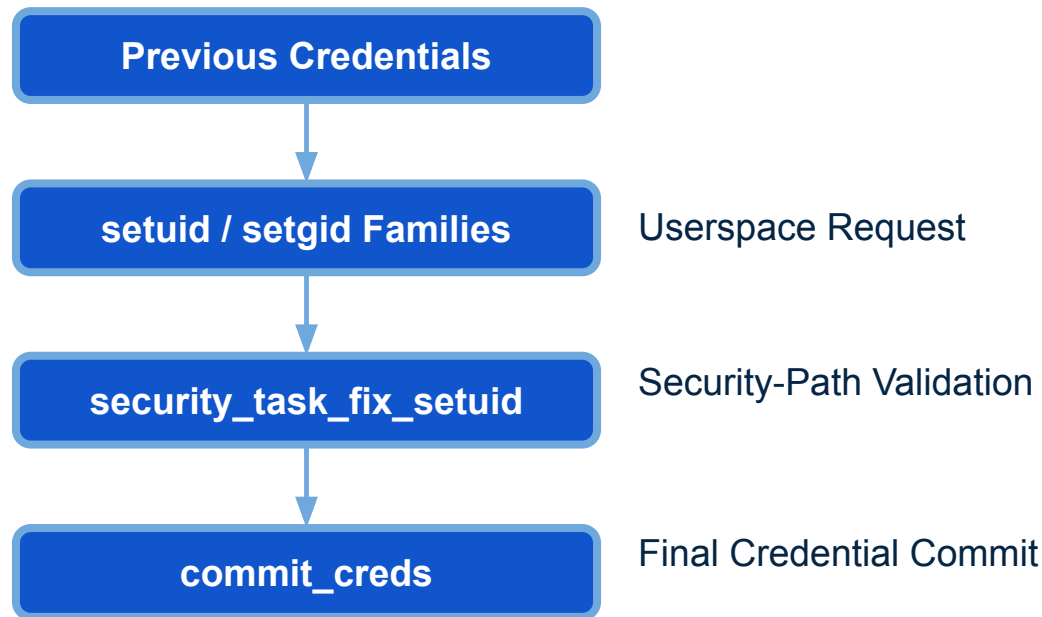
sched_process_fork
**Parent-child
relationship:**
Observe the task
relationship established
by the kernel.

sched_process_exec
Executable image:
Confirm successful image
replacement.

Execution Monitoring: Attempts and Outcomes



Identity, Credentials, and Capabilities



Capability and process control

- **cap_capable:** Observes checks for specific capabilities.
- **ptrace** and **security_task_prctl:** Observe direct process interaction and state manipulation.

File, Permission, and Filesystem Security

File Access

- **open**: reports the syscall outcome.
- **security_file_open** / **security_file_permission**: expose stable metadata (path, inode, device, and ctime)

Permissions

- The **chmod** and **chown** families monitor permission and ownership changes, including return values.

Inode

- **unlink**, **rename**, **symlink** and **mknod**: track structural filesystem changes.

Mounting

- **security_sb_mount** / **security_sb_unmount**: observe mount points and associated flags.

Memory and Namespaces

Memory Management

mmap

mprotect

pkey_mprotect

memfd_create

process_vm_readv / process_vm_writev

Visibility into executable-memory transitions, fileless execution, unusual mappings, and cross-process memory access.

Namespace Transitions

setns

unshare

switch_task_ns

Observes isolation requests and verifies the namespace changes effectively applied by the kernel.

Kernel Hardening and Modules

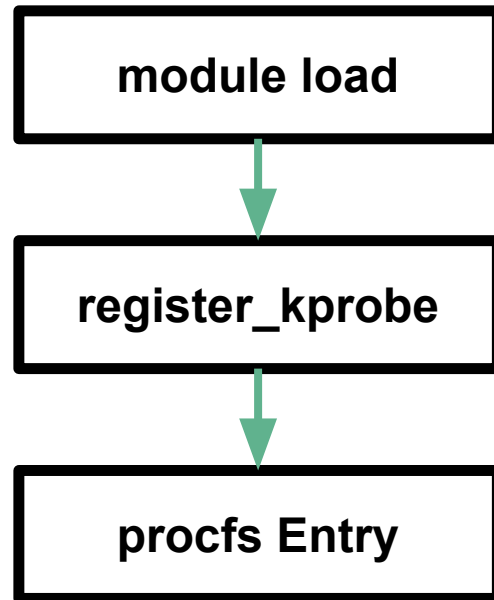
Module lifecycle: *module_load,*
module_free,
do_init_module.

eBPF surface: *security_bpf,*
security_bpf_map,
security_bpf_prog.

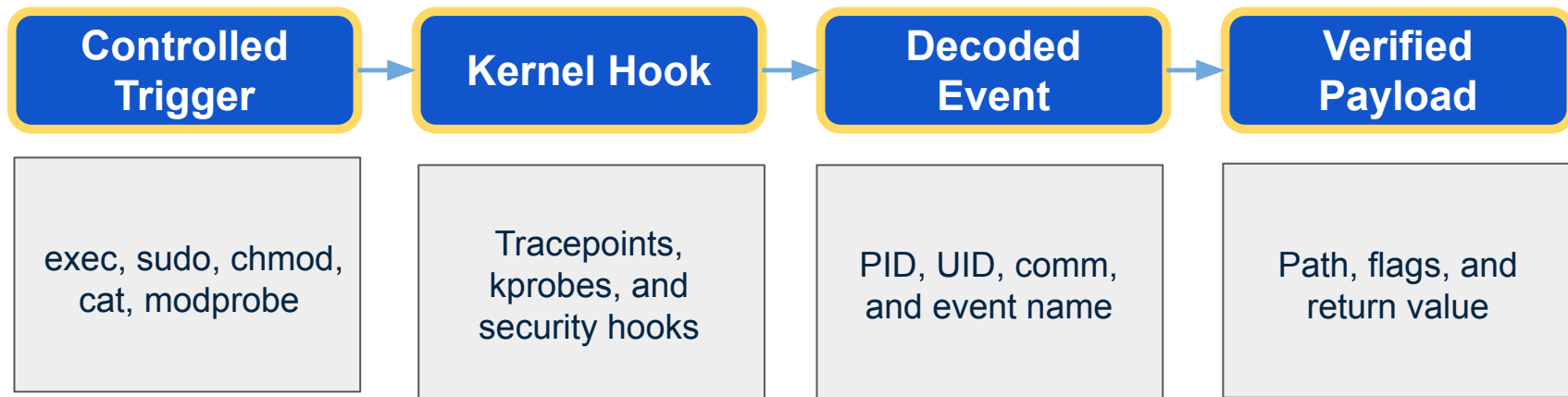
Dynamic instrumentation: *register_kprobe,*
kallsyms_lookup_name.

Kernel interfaces: *proc_create,*
call_usermodehelper,
security_kernel_read_file.

Example of a scenario
observable through distinct
events



Testing and Validation on Rocky Linux 4.18



- **Target:** Less than 5% of one CPU core.
- **CPU** (2-vCPU VM, Kernel 4.18): 5.9%–7.7% steady-state usage.
- **Memory:** Approximately 45 MB RSS.

Current Limitations

- Noise and overhead from some hooks.
- Syscall outcomes are not available for every event.
- Advanced coverage and collective detectors still require consolidation.

Planned Technical Work

- Kernel-side filtering and alert throttling.
- Additional hooks and enter/exit implementations.
- Automated kernel/userspace benchmarks under load.

eBPF Network Monitoring

1. Two collection surfaces

Socket-level security hooks and packet level cgroup/SKB programs.

2. Layered decoding

Kernel-side protocol classification with userspace decoding and enrichment.

3. Process attribution

Every packet is linked to the process and the syscall that produced it.

Process events describe who acts, network events describe where and towards what.

Two Collection Surfaces

Socket-level security hooks

Hook point: kprobes on the kernel's LSM socket hook (*security_socket_**)

Observed data: addresses, ports, families and options requested

Strength: Direct attribution to PID, command, credentials and cgroup

Trade-off: no visibility into the bytes actually transmitted

Blind spot: the bytes actually transmitted

Packet-level

Hook point: Ingress and egress packet path of the monitored cgroup.

Observed data: Packet headers, payload bytes and selected application-layer fields.

Strength: Direct inspection of observed traffic, including DNS, HTTP and TLS handshake metadata.

Trade-off: Payload interpretation depends on encryption, segmentation, fragmentation and protocol support.

Blind spot: the process that produced the packet

Each surface holds exactly what the other is missing

Socket-level Hooks

Hook	What observes	Data submitted to userspace
<code>security_socket_create</code>	Socket creation intent	family,type, protocol, kern flag
<code>security_socket_bind</code>	Local address binding	local sockaddr, port
<code>security_socket_connect</code>	Outbound connection request	remote sockaddr, family, port
<code>security_socket_listen</code>	Passive open	local sockaddr, backlog
<code>security_socket_accept</code>	Inbound connection accepted	local sockaddr
<code>security_socket_setsockopt</code>	Socket option change	level,optname, local sockaddr

Kernel-only support hooks - `sock_alloc_file`, `security_sk_clone`, `security_socket_sendmsg`, `security_socket_recvmsg`: these programs maintain socket-to-process ownership in BPF maps. They never write to the perf buffer.

Packet tracing with cgroup v2

Attach point

Two eBPF programs are attached to a cgroup, one for ingress and one for egress. The kernel runs them for the socket traffic associated with that cgroup and its descendants.

Why cgroup v2

Cgroup v2 provides one unified process hierarchy for all processes. Attaching at the root covers the whole visible cgroup hierarchy.

Cgroup2 mountpoint

The attach operation requires a filesystem path representing the target cgroup. Rocky Linux 8 normally uses cgroup v1, so the runtime creates a temporary cgroup2 mount when none is available.

Packet interception

When eligible socket traffic crosses the ingress or egress path, the kernel invokes the corresponding program and provides a struct `__sk_buff` packet context.

Process ↔ Packet correlation

1. Socket hook

The socket operation runs in process context, so the identity of the caller is directly available.

`pid, tid, uid, cgroup id`

Stored in *inodemap* using the *socket inode* as key

2. Kernel correlation state

A kprobe on `__cgroup_bpf_run_filter_skb` reads the socket inode and recovers the saved context.

It then builds a packet indexer and stores the prepared network event context.

`cgrpctxmap[indexer] = net_event_context`

3. cgroup / skb hook

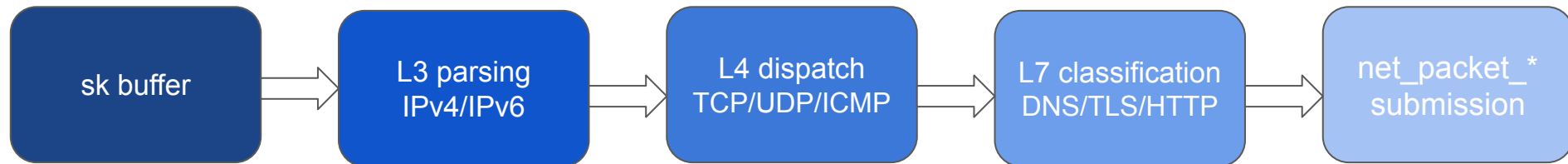
The packet program rebuilds the same indexer and retrieves the same event context.

It adds addresses, ports, protocol and payload metadata.

process context + packet metadata
||
userspace

The socket inode recovers the process context, while the packet indexer carries that context into packet processing.

Kernel-side Packet Parsing and Event Fan-out



One packet may generate multiple layer-specific events



How each layer is identified: L3 by socket family / IP version - L4 by the IP protocol fields - L7 by port + content signature (TLS record and handshake byte, HTTP method)

Benefit: layer-specific events simplify filtering, protocol decoding and detector rules.

Application-Layer Decoding: DNS, HTTP and TLS

DNS

UDP or TCP / port 53

Kernel classification

Traffic identified over UDP or TCP using port 53

Userspace decoding

Extracts questions, answers, records, types and response codes

Metadata

Queried domain, resolved addresses and DNS status

Limitation

DNS over HTTPS or TLS is not visible

HTTP

method or status line

Kernel classification

Request/response method such as POST, GET, PUT and HTTP/

Userspace decoding

Method, host, path or status line when available

Metadata

Readable HTTP context for logs and rules

Limitation

HTTPS payload is not visible; only clear-text HTTP

TLS

handshake record

Kernel classification

Handshake record recognition with ClientHello/ServerHello

Userspace decoding

Custom ClientHello parser extracts the SNI extension

Metadata

Handshake type, TLS version and requested hostname

Limitation

Application payload after the handshake remains encrypted

DNS cache and Network Event Enrichment

DNS answer are used as bounded context to enrich later packet events that only contain an IP address

Cache model:

- **Bounded LRU map**
- **Key:** destination IP
- **Value:** list of hostnames, most recent first. Up to 8

How it works



Advantages:

- Adds domain context to IP-only packet events
- Detectors match on names, not addresses
- Connects DNS activity to later TCP/TLS connections

DNS <-> TLS SNI correlation

DNS answers may return multiple IPs for one name.

TLS ClientHello carries a server-name list; the parser extracts the host entry.

The enriched event can show:

DNS name + destination IP + TLS SNI

Planned Technical Work

- Complete policy integration and activate kernel-side filtering
- Implementation of network detectors.
- Introduce TCP stream reconstruction.
- Extend protocol support and run network-specific benchmarks.